

AFRILANG Stdlib

std/c/statbayes

Bibliothèque AFRILANG `std/c/statbayes` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : bayesUpdate, po

```
`import "std/c/statbayes"` · `use statbayes`
```

- `bayesUpdate(prior number, likelihood number, evidence number) ? number`
- `posteriorOdds(priorOdds number, likelihoodRatio number) ? number`
- `priorFromOdds(odds number) ? number`
- `oddsFromPrior(prior number) ? number`
- `bayesFactor(likelihoodH1 number, likelihoodH0 number) ? number`
- `credibleInterval(samples list number) ? list number`
- `expectedLoss(losses list number, probs list number) ? number`

Category: complex | Tier: complex | Functions: 7

FRANCAIS

1. Vue d'ensemble

Bibliothèque AFRILANG `std/c/statbayes` (niveau complex, catégorie complex). Elle expose 7 fonction(s) : bayesUpdate, po

```
`import "std/c/statbayes" . `use statbayes`  
- `bayesUpdate(prior number, likelihood number, evidence number) ? number`  
- `posteriorOdds(priorOdds number, likelihoodRatio number) ? number`  
- `priorFromOdds(odds number) ? number`  
- `oddsFromPrior(prior number) ? number`  
- `bayesFactor(likelihoodH1 number, likelihoodH0 number) ? number`  
- `credibleInterval(samples list number) ? list number`  
- `expectedLoss(losses list number, probs list number) ? number`
```

2. Installation & import

Ajoutez le module à votre programme :

```
import "std/c/statbayes"  
use statbayes
```

Niveau : complex · Catégorie : Modules complex (c/)
Domaines avancés ? graphes, NLP, réseaux complexes.

3. Référence API

```
? bayesUpdate(prior number, likelihood number, evidence number) ? number  
? posteriorOdds(priorOdds number, likelihoodRatio number) ? number  
? priorFromOdds(odds number) ? number  
? oddsFromPrior(prior number) ? number  
? bayesFactor(likelihoodH1 number, likelihoodH0 number) ? number  
? credibleInterval(samples list number) ? list  
? expectedLoss(losses list number, probs list number) ? number
```

4. Exemples d'utilisation

```
import "std/c/statbayes"  
use statbayes  
  
create result = bayesUpdate(/* args */)   
say result  
  
create result = posteriorOdds(/* args */)   
say result  
  
create result = priorFromOdds(/* args */)   
say result  
  
create result = oddsFromPrior(/* args */)   
say result  
  
create result = bayesFactor(/* args */)   
say result  
  
create result = credibleInterval(/* args */)   
say result
```

5. Bonnes pratiques

```
? Préférez `import "std/c/statbayes"` en tête de fichier.  
? Lancez `afirilang check` avant de livrer.  
? Combinez avec les modules stdlib de la même catégorie.  
? Voir /stdlib/ pour le source live et les démos playground.  
? Signalez les problèmes sur GitHub si une export manque.
```

6. Annexe ? source

```
module statbayes
```

```
function bayesUpdate(prior number, likelihood number, evidence number) returns number
end

function posteriorOdds(priorOdds number, likelihoodRatio number) returns number
end

function priorFromOdds(odds number) returns number
end

function oddsFromPrior(prior number) returns number
end

function bayesFactor(likelihoodH1 number, likelihoodH0 number) returns number
end

function credibleInterval(samples list number) returns list number
end

function expectedLoss(losses list number, probs list number) returns number
end
```

ENGLISH

1. Overview

AFRILANG library `std/c/statbayes` (tier complex, category complex). Exports 7 function(s):
bayesUpdate, posteriorOdds,

```
`import "std/c/statbayes"` · `use statbayes`  
- `bayesUpdate(prior number, likelihood number, evidence number) ? number`  
- `posteriorOdds(priorOdds number, likelihoodRatio number) ? number`  
- `priorFromOdds(odds number) ? number`  
- `oddsFromPrior(prior number) ? number`  
- `bayesFactor(likelihoodH1 number, likelihoodH0 number) ? number`  
- `credibleInterval(samples list number) ? list number`  
- `expectedLoss(losses list number, probs list number) ? number`
```

2. Installation & import

Add the module to your program:

```
import "std/c/statbayes"  
use statbayes
```

Tier: complex · Category: Complex modules (c/)
Advanced domains ? graphs, NLP, complex networks.

3. API reference

```
? bayesUpdate(prior number, likelihood number, evidence number) ? number  
? posteriorOdds(priorOdds number, likelihoodRatio number) ? number  
? priorFromOdds(odds number) ? number  
? oddsFromPrior(prior number) ? number  
? bayesFactor(likelihoodH1 number, likelihoodH0 number) ? number  
? credibleInterval(samples list number) ? list  
? expectedLoss(losses list number, probs list number) ? number
```

4. Usage examples

```
import "std/c/statbayes"  
use statbayes  
  
create result = bayesUpdate(/* args */)   
say result  
  
create result = posteriorOdds(/* args */)   
say result  
  
create result = priorFromOdds(/* args */)   
say result  
  
create result = oddsFromPrior(/* args */)   
say result  
  
create result = bayesFactor(/* args */)   
say result  
  
create result = credibleInterval(/* args */)   
say result
```

5. Best practices

```
? Prefer `import "std/c/statbayes"` at the top of your file.  
? Run `afirang check` before shipping.  
? Combine with related stdlib modules from the same category.  
? See /stdlib/ for the live source and playground demos.  
? Report issues on GitHub if an export is missing or undocumented.
```

6. Appendix ? source

```
module statbayes
```

```
function bayesUpdate(prior number, likelihood number, evidence number) returns number
end

function posteriorOdds(priorOdds number, likelihoodRatio number) returns number
end

function priorFromOdds(odds number) returns number
end

function oddsFromPrior(prior number) returns number
end

function bayesFactor(likelihoodH1 number, likelihoodH0 number) returns number
end

function credibleInterval(samples list number) returns list number
end

function expectedLoss(losses list number, probs list number) returns number
end
```